

Introduction à l'apprentissage profond

TP4

S. Boudra & I. Yahiaoui

Sauvegarder et charger un modèle¹

Sauvegarder les poids et l'architecture dans le même fichier

On peut sauvegarder l'architecture et les poids d'un modèle dans un seul et même fichier H5 avec la fonction `save()`. Ici toutes les informations relatives au modèle sont sauvegardées, à savoir :

- Les poids du modèle.
- L'architecture du modèle.
- Les détails de compilation du modèle (loss et metrics).
- L'état de l'optimizer du modèle (algo qui fait la descent de gradient).

```
# charger la dataset diabetes
# recuperer les données dans X et les labels dans Y
# definir le modele
# compiler le modele
# entrainer le modele
# evaluer le modele
# sauvegarder le model + son archi ds le même fichier
model.save("poids_architecture_modele.h5")
print("Saved model to disk")
```

Pour charger le modèle on utilise la fonction `load_model()`

Ici, il n'est **PAS NECESSAIRE** de compiler le modèle chargé.

```
from keras.models import load_model

# charger le modele
model = load_model('model.h5')
# afficher l'archi du modele
# charger le dataset
# recuperer les données dans X et les labels dans Y
# evaluer le modele
# pas besoin de compiler avant d'evaluer
score = model.evaluate(X, Y, verbose=0)
# afficher l'accuracy
print("%s: %.2f%%" % (model.metrics_names[1], score[1]*100))
```

¹ https://keras.io/getting_started/faq/#what-are-my-options-for-saving-models

Qst : Compléter le code et tester le sur la base *diabetes*

Sauvegarder les poids dans un fichier H5 et l'architecture seule dans un fichier JSON

On peut aussi sauvegarder les poids et l'architecture séparément.

Pour les poids, on les sauvegarde avec la fonction `save_weights()` dans un fichier H5, et l'architecture dans un fichier JSON avec la fonction `to_json()`

```
# charger la dataset diabetes
# recuperer les données dans X et les labels dans Y
# definir le modele
# compiler le modele
# entrainer le modele
# evaluer le modele
# sérialiser le model vers un fichier JSON
model_json = model.to_json()
with open("architecture_modele.json", "w") as json_file:
    json_file.write(model_json)
# sérialiser les poids vers un ficheir HDF5
model.save_weights("poids_modele.h5")
print("Saved model to disk")
```

Pour charger les poids on utilise la fonction `load_weights()`.

Pour charger l'architecture, on utilise la fonction `model_from_json()`

Ici il est **NECESSAIRE** de compiler le modèle chargé **AVANT** de l'évaluer

```
from keras.models import model_from_json
import os

# charger l'archi à partir du fichier JSON et créer le modele
json_file = open('architecture_modele.json', 'r')
loaded_model_json = json_file.read()
json_file.close()
loaded_model = model_from_json(loaded_model_json)
# charger les poids
loaded_model.load_weights("poids_modele.h5")
print("Loaded model from disk")

# il est NECESSAIRE de compiler avant d'evaluer le modele
loaded_model.compile(loss='binary_crossentropy', optimizer='rmsprop', metrics=['accuracy'])
# evaluer le modele
score = loaded_model.evaluate(X, Y, verbose=0)
# afficher l'accuracy du modele
print("%s: %.2f%%" % (loaded_model.metrics_names[1], score[1]*100))
```

Qst : Compléter le code et tester le sur la base *diabetes*

Sauvegarder l'état d'un modèle : checkpoints

Entraîner un modèle d'apprentissage profond peut prendre plusieurs heures voire des jours, il est important de sauvegarder l'état du modèle durant l'entraînement en cas d'arrêt non prévu.

Lors de l'entraînement, on peut sauvegarder l'état du modèle afin de déterminer l'ensemble des paramètres (les poids) à une epoch précise qui retournent les meilleures performances. Ceci est réalisable via la classe `ModelCheckpoint`² dont l'objet instancié est passé à l'attribut `callbacks` de `fit()`.

```
from keras.callbacks import ModelCheckpoint

# charger la base de donnees diabetes
# definir le modele
# compiler le modele
# checkpoint
filepath="weights-improvement-{epoch:02d}-{val_loss:.2f}.hdf5"
checkpoint = ModelCheckpoint(filepath, monitor='val_loss', verbose=1,
                             save_best_only=True, mode='min')
callbacks_list = [checkpoint]
# entrainer le modele
model.fit(X, Y, validation_split=0.33, epochs=150, batch_size=10,
          callbacks=callbacks_list, verbose=0)
```

A l'exécution, un fichier .hdf5 est sauvegardé à chaque amélioration de la valeur de l'accuracy.

Aussi, on peut sauvegarder seulement les meilleurs poids pour la valeur d'accuracy la plus élevée lors de l'entraînement du modèle. Il suffit juste de passer le nom d'un seul fichier .hdf5.

```
from keras.callbacks import ModelCheckpoint

# charger la base de donnees diabetes
# definir le modele
# compiler le modele
# checkpoint
filepath="weights-best.hdf5"
checkpoint = ModelCheckpoint(filepath, monitor='val_loss', verbose=1,
                             save_best_only=True, mode='min')
callbacks_list = [checkpoint]
# entrainer le modele
model.fit(X, Y, validation_split=0.33, epochs=150, batch_size=10,
          callbacks=callbacks_list, verbose=0)
```

On peut charger les poids sauvegardés pour les utiliser plus tard sans avoir à entraîner à nouveau le modèle. Pour cela on utilise la fonction `load_weights()` du modèle (revoir la section « sauvegarder et charger un modèle en Keras »).

Qst : Compléter le code et tester le sur la base *diabetes*

² https://keras.io/api/callbacks/model_checkpoint/